Carbook on Oracle Cloud (Always Free Arm VM) with Coolify

Target: `https://analysight.com` (canonical; `www.analysight.com` redirects to it) on a `VM.Standard.A1.Flex` VM (Ubuntu, arm64) running Coolify. **Source:** `carbook/` in `atifmahmoodai/web-automotive-apps`, deployed from branch **`main`** **(Carbook 0.2.0)** once this PR is merged. The `mobile/` app is not part of this deployment. **Sign-in:** GitHub, through oauth2-proxy, limited to GitHub user **`atifmahmoodai`** by default.

Your checklist (things only you can do)

1. **Oracle VM:** create the `VM.Standard.A1.Flex` instance (Ubuntu 22.04/24.04 arm64, for example 4 OCPU/24 GB) and **reserve its public IP**.
2. **Open ports:**
   - In the VCN security list, allow ingress TCP **22, 80, 443**. Allow **8000** only while you set up Coolify.
   - On the VM, run `sudo iptables -I INPUT -p tcp -m multiport --dports 80,443,8000 -j ACCEPT && sudo netfilter-persistent save`.
3. **DNS at your registrar for analysight.com:**

Type	Name	Value
A	`@`	VM reserved public IP
A	`www`	VM reserved public IP
A	`coolify` (optional)	VM reserved public IP, for the Coolify dashboard

   If the DNS is on Cloudflare, use **DNS only** (grey cloud).
4. **Install Coolify** on the VM with `curl -fsSL https://cdn.coollabs.io/coolify/install.sh | sudo bash`, open `http://VM_IP:8000`, and create the admin account. Optionally serve the dashboard at `https://coolify.analysight.com`, then close port 8000.
5. **Create a GitHub OAuth App** (GitHub → Settings → Developer settings → OAuth Apps → New):
   - Homepage URL: `https://analysight.com`
   - Authorization callback URL: **`https://analysight.com/oauth2/callback`**
   - Copy the **Client ID**, then generate a **Client secret**.
6. **Generate a cookie secret** with `openssl rand -base64 32 | tr -- '+/' '-_'`.
7. **Merge this PR into `main`**, or point Coolify at the `carbook-oracle-deploy` branch.
8. **In Coolify, create the resource:** New Resource → Public Repository (or GitHub App) → `atifmahmoodai/web-automotive-apps`, with these settings:

- Branch: `main`
- Build Pack: **Docker Compose**
- Base Directory: **`/carbook`**
- Docker Compose Location: **`/docker-compose.yml`**
- Domains: on the **`auth`** service only, enter `https://analysight.com:4180,https://www.analysight.com:4180`. Leave `carbook` with no domain.
- Keep "Escape special characters in labels" at its default (on).

9. **Set environment variables in Coolify** (names only; you enter the values):

Name	Value
`OAUTH2_PROXY_CLIENT_ID`	GitHub OAuth App Client ID
`OAUTH2_PROXY_CLIENT_SECRET`	GitHub OAuth App Client secret
`OAUTH2_PROXY_COOKIE_SECRET`	the value from step 6
`PUBLIC_HOST` *(optional)*	defaults to `analysight.com`
`OAUTH2_PROXY_GITHUB_USERS` *(optional)*	defaults to `atifmahmoodai`; comma-separate extra usernames to invite people

10. **Deploy,** then open `https://analysight.com` and **sign in with GitHub first.** The first account to sign in becomes the community owner.

11. **Check:** `https://www.analysight.com` should redirect (301) to `https://analysight.com`.

12. **Backups:** turn on Oracle block or boot volume backups. The Docker volume `carbook-data` holds everything: `/data/v3/d1` is the SQLite database and `/data/v3/r2` holds uploads.

How it works

- **Carbook is a single Cloudflare Worker.** It is built with Vinext (a Next.js-style framework on Vite) and serves both the pages and the same-origin API (`/api/state`, `/api/read`, `/api/action`, `/api/upload`, `/api/media/:id`, `/api/mobile-link`). It uses two bindings: **D1** (`DB`, SQLite, Drizzle migrations in `drizzle/`) and **R2** (`BUCKET`, for photos and videos). It uses no KV, Durable Objects or secrets.
- **Chosen approach:** run the built Worker unchanged in **workerd**, Cloudflare's open-source runtime, via `wrangler dev --local`. workerd has native linux-arm64 builds. D1 becomes local SQLite and R2 becomes local files on the `carbook-data` volume. Migrations run automatically at every start, and ones already applied are skipped.
- **Login layer:**
  - The app was written for ChatGPT "Sites" hosting, which adds `oai-authenticated-user-*` headers. Self-hosted, those headers could be faked.
  - So **oauth2-proxy** (the `auth` service) handles GitHub login and only lets `OAUTH2_PROXY_GITHUB_USERS` in.
  - After login it sets `X-Forwarded-User` / `X-Forwarded-Email` and strips any copies the client sends.

- **Code change** (`carbook/worker/index.ts` , `carbook/cloudflare-env.d.ts` , also kept as `self-hosted-auth.patch`): when `SELF_HOSTED=1` , the Worker drops any browser-sent `oai-*` headers and sets them from the proxy's verified user. It also redirects `/signin-with-chatgpt` to the page you came from and `/signout-with-chatgpt` to `/oauth2/sign_out` . On Sites/Cloudflare, where `SELF_HOSTED` isn't set, the change does nothing.
- **Canonical domain:** a Traefik `redirectregex` middleware on the `auth` service, attached with `coolify.traefik.middlewares` , sends `www.*` to the apex with a 301. The app's same-origin checks need one public origin. The Worker is started with `--local-upstream analysight.com --upstream-protocol https` so that its request URL matches that origin.
- **Alternatives rejected:**
  - Porting to Node with SQLite and disk storage means rewriting every D1 and R2 call, and Vinext has no stable Node target.
  - "Frontend on the VM, Worker on Cloudflare" isn't possible because the frontend and backend are one Worker.
- **Tradeoff:** `wrangler dev` is a development server running a single instance. It's fine for a private or small community, not tuned for heavy public traffic. It uses about 300–500 MB RAM.

Files

Path	Purpose
`carbook/Dockerfile`	Two-stage build on `node:22-bookworm-slim` (arm64 or amd64; glibc is needed by workerd). pnpm 11.25.0 build from the lockfile, then a runtime with `wrangler@4.92.0`
`carbook/docker-entrypoint.sh`	Applies D1 migrations, then starts workerd on `:8787`
`carbook/docker-compose.yml`	`carbook` (internal only, volume `carbook-data`) and `auth` (oauth2-proxy v7.15.6 on `:4180` , the www redirect)
`carbook/.dockerignore`	Keeps `node_modules` , `mobile/` and build output out of the build context
`carbook/deploy/oracle/`	This guide (`DEPLOY.md` , PDF) and reference copies of the files above, plus `self-hosted-auth.patch` (already applied)

Updating and operations

- To update, push to `main` ; Coolify can auto-deploy on push, or you can click Redeploy.
- Upload limits are set by the app: 5 MB per photo and 20 MB per video.
- To let more people in, add their GitHub usernames to `OAUTH2_PROXY_GITHUB_USERS` and redeploy.

What was tested (on a build box, 10–11 Nov 2026)

- **Docker build:** a full `docker build` of `carbook/` on linux/amd64 completed. The arm64 runtime image ran under QEMU using the native `workerd-linux-arm64` binary, and the app worked. On the Oracle VM the whole build runs natively on arm64.
- **Smoke test, run after the patch was applied:**
  - migrations apply and the home page returns 200;

    - spoofed `oai-*` headers are ignored, and proxy headers sign the user in;
    - the first user becomes owner;
    - same-origin POSTs are accepted and cross-origin POSTs are rejected;
    - photo upload and download return 200;
    - the sign-in and sign-out redirects work;
    - data survives a container re-create, and the healthcheck reports healthy.
- **oauth2-proxy v7.15.6:** unauthenticated and header-spoofed requests are redirected to GitHub login with callback `https://analysight.com/oauth2/callback`.
- **Not tested:** a real GitHub login, Coolify itself, the Traefik www redirect, and a native arm64 build.